MOD-07: Autonomy SW

Technical Documentation

Name 1:	Ayşe Feyza SERBEST
Name 2:	Görkem UYSAL

Version:	0.2
Date:	March 29, 2026

Contents

1 Purpose **2**

2 Dependencies **2**

3 FSM Design **2**
3.1 State Enumeration 2
3.2 Transition Diagram 3
3.3 State Descriptions 4

4 Data Types **4**
4.1 Input Structure 4
4.2 Output Structure 5

5 API Summary **5**

6 Operational Flow **5**

7 Known Limitations **6**

8 Version History **6**

1. Purpose

MOD-07 (Autonomy SW) is the central decision-making layer of the Autonomous Fire Detection and Suppression Rover. It implements a **Finite State Machine (FSM)** that integrates sensor data from multiple modules and issues coordinated commands to all actuator subsystems.

No other module contains mission-level logic. All remaining modules are either hardware drivers or software abstractions over hardware. MOD-07 is the sole component that determines *what the rover should be doing at any given moment*, making it the architectural apex of the system.

The module's responsibilities are:

- Consuming structured flame detection data from MOD-02 (`flame_data_t`).
- Consuming real-time proximity data from MOD-05 (`distance_cm`).
- Transitioning the FSM across seven defined states based on sensor conditions.
- Issuing motion commands to MOD-04 (`motion_direction_t`, `speed`).
- Issuing suppression commands to MOD-06 (`suppression_cmd_t`).
- Exposing its current state to MOD-08 for testing and validation.

2. Dependencies

Module	Name	Usage
MOD-02	Flame Detection	Flame direction / intensity input
MOD-04	Motion Control	Receives movement commands
MOD-05	Distance Sensor	Obstacle / range input
MOD-06	Fire Suppression	Pump activation output
MOD-08	Test Module	FSM verification

Table 1: Dependencies of MOD-07

3. FSM Design

3.1 State Enumeration

```
typedef enum {
STATE_SEARCH,
STATE_DETECT,
STATE_APPROACH,
STATE_AVOID_OBSTACLE,
STATE_STOP,
STATE_EXTINGUISH,
STATE_VERIFY
} autonomy_state_t;
```

Listing 1: FSM States

3.2 Transition Diagram

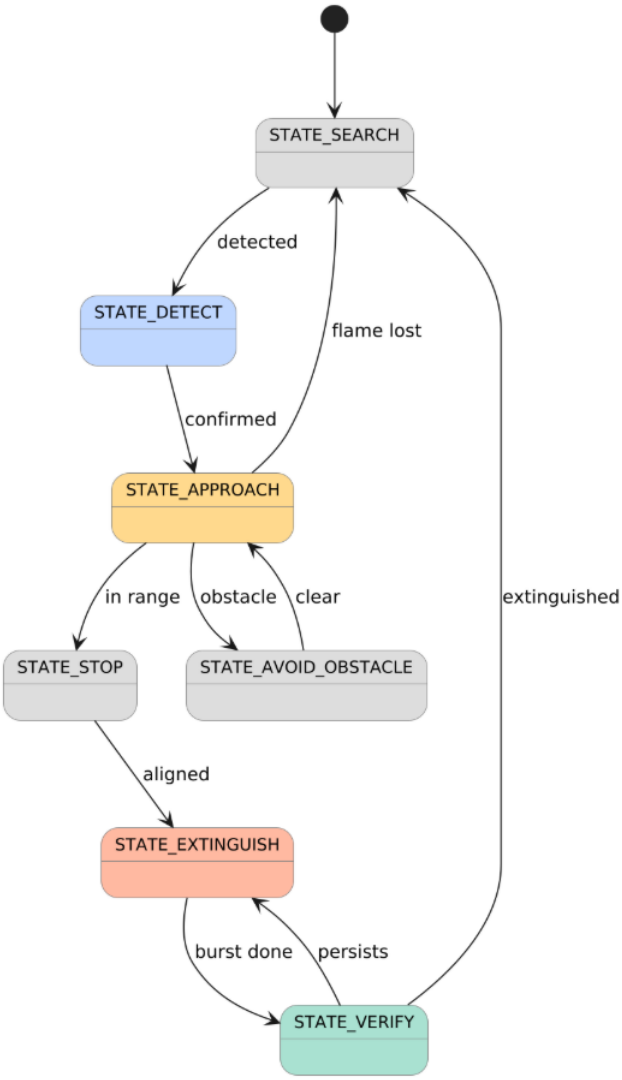

Figure 1: FSM State Transition Diagram

3.3 State Descriptions

Table 2: FSM state descriptions and trigger conditions

State	Actuators Active	Description & Exit Condition
`STATE_SEARCH`	MOD-04 (rotate)	Rover rotates in place, polling MOD-02 continuously. Exits when `flame_data_t.detected == true`.
`STATE_DETECT`	None	Flame signal confirmed; direction is resolved. Exits immediately to `STATE_APPROACH` once direction is stable.
`STATE_APPROACH`	MOD-04 (drive)	Rover drives toward flame using directional commands. MOD-05 is polled every cycle. Exits to `STATE_STOP` when `distance_is_unsafe()` returns true, or to `STATE_AVOID_OBSTACLE` on lateral obstacle detection. Exits to `STATE_SEARCH` if flame is lost.
`STATE_AVOID_OBSTACLE`	MOD-04 (steer)	Rover executes avoidance maneuver. Exits back to `STATE_APPROACH` when path is clear.
`STATE_STOP`	None	All motors halted; rover aligns with flame. Exits to `STATE_EXTINGUISH`.
`STATE_EXTINGUISH`	MOD-06 (pump on)	Water pump is activated via `suppression_cmd_t`. MOD-02 is polled to detect flame extinction. Exits to `STATE_VERIFY` after pump burst completes.
`STATE_VERIFY`	None	Confirms `flame_data_t.detected == false`. Exits to `STATE_SEARCH` if flame persists; exits to `STATE_STOP` (idle) if extinguished.

4. Data Types

4.1 Input Structure

```
typedef struct {
    flame_data_t flame_data;
    uint16_t distance_cm;
} autonomy_input_t;
```

Listing 2: autonomy$_input_t$

Field	Type	Description
flame_data	flame_data_t	Flame direction/intensity data
distance_cm	uint16_t	Distance in cm

Table 3: Input Structure Fields

4.2 Output Structure

```
typedef struct {
    motion_direction_t motion_cmd;
    uint8_t speed;
    uint8_t pump_on;
} autonomy_output_t;
```

Listing 3: autonomy$_output_t$

Field	Type	Description
motion_cmd	motion_direction_t	Direction command
speed	uint8_t	PWM speed (0-255)
pump_on	uint8_t	Pump activation flag

Table 4: Output Structure Fields

5. API Summary

Function	Description
autonomy_init()	Initializes FSM
autonomy_update(in)	Updates FSM with sensor input
autonomy_get_output(out)	Returns current commands
autonomy_reset()	Resets FSM
autonomy_get_current_state()	Returns current state

Table 5: Public API

6. Operational Flow

1. Search for flame
2. Detect target
3. Move toward flame
4. Avoid obstacles if needed
5. Stop at safe distance
6. Activate pump

7. Verify extinguished
8. Return to search mode

7. Known Limitations

- No advanced path planning
- No timeout watchdog
- No speed ramping
- Sensor noise may affect transitions

8. Version History

Version	Date	Notes
0.1	2026-03-28	Initial FSM created
0.2	2026-03-29	Added new states and robustness

Table 6: Version History